Algorithms and Data Structures

Sorting, Complexity, and Queues

Martin Benning

University College London

ENGF0034 – Design and Professional Skills

Lecture Overview

Objectives

To explore the distinction between Abstract Data Types (ADTs) and concrete Data Structures, implement and analyse the Queue ADT, and delve deeply into the Sorting problem, focusing on the Merge Sort algorithm, its complexity, and its properties.

ADTs, Data Structures, and Complexity

Organising Data: Abstraction vs Implementation (I)

Abstract Data Type (ADT)

A mathematical model for data types, defined by its behaviour (semantics) from the point of view of the user.

- Defines **what** operations are supported (the interface).
- Examples: List, Stack, Queue, Set, Map.

Data Structure

A concrete way of organizing data in memory to implement an ADT efficiently.

- Defines how data is stored and operations are implemented.
- Examples: Array (Python 'list'), Linked List, Hash Table, Tree.

Organising Data: Abstraction vs Implementation (I)

Abstract Data Type (ADT)

A mathematical model for data types, defined by its behaviour (semantics) from the point of view of the user.

- Defines **what** operations are supported (the interface).
- Examples: List, Stack, Queue, Set, Map.

Data Structure

A concrete way of organizing data in memory to implement an ADT efficiently.

- Defines **how** data is stored and operations are implemented.
- Examples: Array (Python 'list'), Linked List, Hash Table, Tree.

Organising Data: Abstraction vs Implementation (I)

Abstract Data Type (ADT)

A mathematical model for data types, defined by its behaviour (semantics) from the point of view of the user.

- Defines **what** operations are supported (the interface).
- Examples: List, Stack, Queue, Set, Map.

Data Structure

A concrete way of organizing data in memory to implement an ADT efficiently.

- Defines **how** data is stored and operations are implemented.
- Examples: Array (Python 'list'), Linked List, Hash Table, Tree.

Organising Data: Abstraction vs Implementation (I)

Abstract Data Type (ADT)

A mathematical model for data types, defined by its behaviour (semantics) from the point of view of the user.

- Defines **what** operations are supported (the interface).
- Examples: List, Stack, Queue, Set, Map.

Data Structure

A concrete way of organizing data in memory to implement an ADT efficiently.

- Defines **how** data is stored and operations are implemented.
- Examples: Array (Python 'list'), Linked List, Hash Table, Tree.

Organising Data: Abstraction vs Implementation (II)

Data Structure

A concrete way of organizing data in memory to implement an ADT efficiently.

- Defines **how** data is stored and operations are implemented.
- Examples: Array (Python ‘list‘), Linked List, Hash Table, Tree.

The Importance of Abstraction

Separating the interface from the implementation allows us to manage complexity and change the underlying structure (e.g., for better performance) without altering the client code.

Analysing Efficiency: Big O Notation

We measure the efficiency of algorithms and data structures based on how resource usage (time/space) scales with the input size N.

Big O Notation ($\mathcal{O}$)

Provides an asymptotic upper bound on the growth rate. Describes the worst-case scenario.

Common Complexities

$\mathcal{O}(1)$ (Constant): Execution time is independent of N. Ideal.

$\mathcal{O}(\log N)$ (Logarithmic): Time increases very slowly (e.g., Binary Search).

$\mathcal{O}(N)$ (Linear): Time is proportional to N.

$\mathcal{O}(N \log N)$ (Log-linear): Common in efficient sorting algorithms.

$\mathcal{O}(N^2)$ (Quadratic): Time grows rapidly; often too slow for large N.

Analysing Efficiency: Big O Notation

We measure the efficiency of algorithms and data structures based on how resource usage (time/space) scales with the input size N.

Big O Notation ($\mathcal{O}$)

Provides an asymptotic upper bound on the growth rate. Describes the worst-case scenario.

Common Complexities

$\mathcal{O}(1)$ (Constant): Execution time is independent of N. Ideal.

$\mathcal{O}(\log N)$ (Logarithmic): Time increases very slowly (e.g., Binary Search).

$\mathcal{O}(N)$ (Linear): Time is proportional to N.

$\mathcal{O}(N \log N)$ (Log-linear): Common in efficient sorting algorithms.

$\mathcal{O}(N^2)$ (Quadratic): Time grows rapidly; often too slow for large N.

Analysing Efficiency: Big O Notation

We measure the efficiency of algorithms and data structures based on how resource usage (time/space) scales with the input size N.

Big O Notation ($\mathcal{O}$)

Provides an asymptotic upper bound on the growth rate. Describes the worst-case scenario.

Common Complexities

$\mathcal{O}(1)$ (Constant): Execution time is independent of N. Ideal.

$\mathcal{O}(\log N)$ (Logarithmic): Time increases very slowly (e.g., Binary Search).

$\mathcal{O}(N)$ (Linear): Time is proportional to N.

$\mathcal{O}(N \log N)$ (Log-linear): Common in efficient sorting algorithms.

$\mathcal{O}(N^2)$ (Quadratic): Time grows rapidly; often too slow for large N.

Analysing Efficiency: Big O Notation

We measure the efficiency of algorithms and data structures based on how resource usage (time/space) scales with the input size N.

Big O Notation ($\mathcal{O}$)

Provides an asymptotic upper bound on the growth rate. Describes the worst-case scenario.

Common Complexities

$\mathcal{O}(1)$ (Constant): Execution time is independent of N. Ideal.

$\mathcal{O}(\log N)$ (Logarithmic): Time increases very slowly (e.g., Binary Search).

$\mathcal{O}(N)$ (Linear): Time is proportional to N.

$\mathcal{O}(N \log N)$ (Log-linear): Common in efficient sorting algorithms.

$\mathcal{O}(N^2)$ (Quadratic): Time grows rapidly; often too slow for large N.

Analysing Efficiency: Big O Notation

We measure the efficiency of algorithms and data structures based on how resource usage (time/space) scales with the input size N.

Big O Notation ($\mathcal{O}$)

Provides an asymptotic upper bound on the growth rate. Describes the worst-case scenario.

Common Complexities

$\mathcal{O}(1)$ (Constant): Execution time is independent of N. Ideal.

$\mathcal{O}(\log N)$ (Logarithmic): Time increases very slowly (e.g., Binary Search).

$\mathcal{O}(N)$ (Linear): Time is proportional to N.

$\mathcal{O}(N \log N)$ (Log-linear): Common in efficient sorting algorithms.

$\mathcal{O}(N^2)$ (Quadratic): Time grows rapidly; often too slow for large N.

Contiguous vs Linked Structures

The choice of data structure involves trade-offs, often related to memory layout.

Arrays (Contiguous Memory)

Elements stored sequentially in memory. (e.g., Python 'list').

- **Pros:** Fast random access ($\mathcal{O}(1)$ indexing). Cache-friendly (locality of reference).
- **Cons:** Insertion/deletion at the front/middle is slow ($\mathcal{O}(N)$) as elements must be shifted.

Linked Lists (Non-Contiguous Memory)

Elements (Nodes) stored individually, connected by pointers (references).

- **Pros:** Dynamic size. Fast insertion/deletion at known locations ($\mathcal{O}(1)$) – no shifting required.
- **Cons:** Slow random access (requires traversal, $\mathcal{O}(N)$). Higher memory overhead for pointers.

Contiguous vs Linked Structures

The choice of data structure involves trade-offs, often related to memory layout.

Arrays (Contiguous Memory)

Elements stored sequentially in memory. (e.g., Python 'list').

- **Pros:** Fast random access ($\mathcal{O}(1)$ indexing). Cache-friendly (locality of reference).
- **Cons:** Insertion/deletion at the front/middle is slow ($\mathcal{O}(N)$) as elements must be shifted.

Linked Lists (Non-Contiguous Memory)

Elements (Nodes) stored individually, connected by pointers (references).

- **Pros:** Dynamic size. Fast insertion/deletion at known locations ($\mathcal{O}(1)$) – no shifting required.
- **Cons:** Slow random access (requires traversal, $\mathcal{O}(N)$). Higher memory overhead for pointers.

The Queue ADT and Implementations

The Queue ADT

Definition

A Queue is a linear data structure where additions occur at one end (the rear or tail) and removals occur at the other end (the front or head).

The Principle: First-In, First-Out (FIFO)

The element that has been in the queue the longest is the first one to be removed.

Core Operations and Goals

Enqueue (Add): Add an element to the tail. Goal: $\mathcal{O}(1)$.

Dequeue (Pop): Remove and return the element from the head. Goal: $\mathcal{O}(1)$.

IsEmpty: Check if the queue is empty. Goal: $\mathcal{O}(1)$.

The Queue ADT

Definition

A Queue is a linear data structure where additions occur at one end (the rear or tail) and removals occur at the other end (the front or head).

The Principle: First-In, First-Out (FIFO)

The element that has been in the queue the longest is the first one to be removed.

Core Operations and Goals

Enqueue (Add): Add an element to the tail. Goal: $\mathcal{O}(1)$.

Dequeue (Pop): Remove and return the element from the head. Goal: $\mathcal{O}(1)$.

IsEmpty: Check if the queue is empty. Goal: $\mathcal{O}(1)$.

The Queue ADT

Definition

A Queue is a linear data structure where additions occur at one end (the rear or tail) and removals occur at the other end (the front or head).

The Principle: First-In, First-Out (FIFO)

The element that has been in the queue the longest is the first one to be removed.

Core Operations and Goals

Enqueue (Add): Add an element to the tail. Goal: $\mathcal{O}(1)$.

Dequeue (Pop): Remove and return the element from the head. Goal: $\mathcal{O}(1)$.

IsEmpty: Check if the queue is empty. Goal: $\mathcal{O}(1)$.

The Queue ADT

The Principle: First-In, First-Out (FIFO)

The element that has been in the queue the longest is the first one to be removed.

Core Operations and Goals

Enqueue (Add): Add an element to the tail. Goal: $\mathcal{O}(1)$.

Dequeue (Pop): Remove and return the element from the head. Goal: $\mathcal{O}(1)$.

IsEmpty: Check if the queue is empty. Goal: $\mathcal{O}(1)$.

Applications

Task scheduling (OS), network buffers, Breadth-First Search (BFS).

Implementation Strategy: The Pitfall of Arrays

A naive approach uses a Python `list` (dynamic array).

```
# Naive Implementation using Python list
q = []

# Enqueue: Add to the end (Tail)
q.append(1) # O(1) amortised

# Dequeue: Remove from the front (Head)
item = q.pop(0) # O(N)
```

The Performance Problem

'pop(0)' is an $\mathcal{O}(N)$ operation. When the first element is removed from an array, all remaining $N-1$ elements must be shifted one position to the left in memory. This violates goal of $\mathcal{O}(1)$.

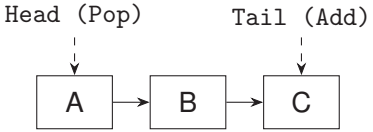
Implementation Strategy: Linked Lists

To achieve $\mathcal{O}(1)$ operations at both ends, we utilise a Linked List.

The Strategy: Head and Tail Pointers

We must maintain references to both the first and the last node.

- **Head:** Used for removal (Dequeue). Removal from the head is $\mathcal{O}(1)$.
- **Tail:** Used for addition (Enqueue). Without this, finding the end would take $\mathcal{O}(N)$. With it, addition is $\mathcal{O}(1)$.



The Node Structure (SimpleNode.py)

We first define the building block: the ‘Node‘.

```
1  class Node():
2      def __init__(self, value):
3          self.value = value
4          self.next = None
5
6      def append(self, node):
7          if self.next is not None:
8              raise(ValueError("next node is not none"))
9          self.next = node
```

Design Note

The provided ‘append‘ method is restrictive, ensuring ‘next‘ is only set once. This simplifies the Queue logic but is not standard for a generic Node class.

The Node Structure (SimpleNode.py)

```
1  class Node():
2      def __init__(self, value):
3          self.value = value
4          self.next = None
5
6      def append(self, node):
7          if self.next is not None:
8              raise(ValueError("next node is not none"))
9          self.next = node
```

Design Note

The provided ‘append‘ method is restrictive, ensuring ‘next‘ is only set once. This simplifies the Queue logic but is not standard for a generic Node class.

Queue Implementation: Initialisation (queue.py)

```
1  class Queue():
2      def __init__(self):
3          # create a new queue
4          self.head = None
5          self.tail = None
6
7      def is_empty(self):
8          # The queue is empty if and only if the head is None
9          return self.head is None
```

Class Invariant

An essential invariant: If the queue is empty, both ‘head‘ and ‘tail‘ must be ‘None‘.

Queue Implementation: Initialisation (queue.py)

```
1  class Queue():
2      def __init__(self):
3          # create a new queue
4          self.head = None
5          self.tail = None
6
7      def is_empty(self):
8          # The queue is empty if and only if the head is None
9          return self.head is None
```

Class Invariant

An essential invariant: If the queue is empty, both ‘head‘ and ‘tail‘ must be ‘None‘.

Queue Implementation: Enqueue (Add)

Adding a new element to the tail. Complexity: $\mathcal{O}(1)$.

```
    def add(self, value):
        # 1. Create the new node
        n = Node(value)

        # 2. Handle the edge case: Empty queue
        if self.head is None:
            self.head = n
        else:
            # 3. Link the current tail to the new node
            self.tail.append(n) # self.tail.next = n

        # 4. Update the tail pointer (in both cases)
        self.tail = n
```

Queue Implementation: Dequeue (Pop)

Removing the element from the head. Complexity: $\mathcal{O}(1)$.

```
    def pop(self):
        # 1. Error handling: Check if empty
        if self.head is None:
            raise(ValueError("Attempt to pop from empty queue"))
        # 2. Store the value and update head pointer
        value = self.head.value
        self.head = self.head.next
        # 3. Special Case: If we removed the last item
        if self.head is None:
            # Must also update tail to maintain the invariant!
            self.tail = None
        return value
```

Queue Implementation: Dequeue (Pop) (cont.)

The Critical Edge Case (Step 3)

If the last element is popped, ‘self.head‘ becomes ‘None‘. Forgetting to also set ‘self.tail = None‘ leads to a ”dangling tail pointer” bug, where the tail points to a node no longer in the queue.

Python Best Practice: `collections.deque`

While implementing data structures manually is crucial for understanding, production Python code should utilise optimised built-in libraries.

`collections.deque` (Double-Ended Queue)

Highly optimised for fast appends and pops on either end ($\mathcal{O}(1)$ amortised).

Using ‘deque‘ as a Queue

```
1 from collections import deque
2 q = deque()
3 q.append(1)       # Enqueue (Add to the right)
4 item = q.popleft() # Dequeue (Remove from the left)
```

The Sorting Problem and Complexity

The Sorting Problem

Sorting is arguably the most fundamental algorithmic problem in computer science.

Formal Definition

Input: A sequence of N elements $A = \langle a_1, a_2, \ldots, a_N \rangle$ and a total order relation $\leqslant$.
Output: A permutation (reordering) $A' = \langle a'_1, a'_2, \ldots, a'_N \rangle$ such that $a'_1 \leqslant a'_2 \leqslant \cdots \leqslant a'_N$.

Why Sort?

- Enables efficient searching (Binary Search in $\mathcal{O}(\log N)$ time).
- Prerequisite for many other algorithms (e.g., finding duplicates, median calculation, database joins).

The Sorting Problem

Sorting is arguably the most fundamental algorithmic problem in computer science.

Formal Definition

Input: A sequence of N elements $A = \langle a_1, a_2, \ldots, a_N \rangle$ and a total order relation $\leqslant$.
Output: A permutation (reordering) $A' = \langle a'_1, a'_2, \ldots, a'_N \rangle$ such that $a'_1 \leqslant a'_2 \leqslant \cdots \leqslant a'_N$.

Why Sort?

- Enables efficient searching (Binary Search in $\mathcal{O}(\log N)$ time).
- Prerequisite for many other algorithms (e.g., finding duplicates, median calculation, database joins).

Sorting: The Problem vs The Algorithm (I)

Sorting is an Operation, Not an Algorithm

It's crucial to distinguish between the **problem** of sorting and the many different **algorithms** that can solve it.

- **The Problem:** Given a collection of items, arrange them into a specific order.
- **The Algorithms:** The specific, step-by-step procedures used to achieve this arrangement (e.g., Bubble Sort, Merge Sort).

Sorting: The Problem vs The Algorithm (II)

An Alternative Formulation: Linear Optimisation

To illustrate this distinction, sorting can be formulated as a mathematical optimisation problem, completely detached from any procedural steps.

- **Goal:** Find a permutation matrix P that sorts a vector x in ascending order.
- **Formulation:** Maximise the inner product of the permuted vector Px with a vector of increasing weights w:

$$\max_{P \in \mathbb{R}^{n \times n}} \langle w, Px \rangle$$

- Subject to the constraints that P is a doubly stochastic matrix:

$$P_{i,j} \in [0, 1], \quad \sum_{i=1}^{n} P_{ij} = 1, \quad \sum_{j=1}^{n} P_{ij} = 1 \quad \forall i, j \in \{1, ..., n\}.$$

Properties of Sorting Algorithms

We classify sorting algorithms based on several criteria beyond time complexity.

In-Place: Does the algorithm use only a constant amount of extra memory ($\mathcal{O}(1)$ space) beyond the input array?

Stability: Does the algorithm preserve the relative order of elements with equal keys?

Adaptivity: Does the algorithm run faster if the input is already partially sorted?

Properties of Sorting Algorithms

We classify sorting algorithms based on several criteria beyond time complexity.

In-Place: Does the algorithm use only a constant amount of extra memory ($\mathcal{O}(1)$ space) beyond the input array?

Stability: Does the algorithm preserve the relative order of elements with equal keys?

Adaptivity: Does the algorithm run faster if the input is already partially sorted?

Properties of Sorting Algorithms

We classify sorting algorithms based on several criteria beyond time complexity.

In-Place: Does the algorithm use only a constant amount of extra memory ($\mathcal{O}(1)$ space) beyond the input array?

Stability: Does the algorithm preserve the relative order of elements with equal keys?

Adaptivity: Does the algorithm run faster if the input is already partially sorted?

Properties of Sorting Algorithms

We classify sorting algorithms based on several criteria beyond time complexity.

In-Place: Does the algorithm use only a constant amount of extra memory ($\mathcal{O}(1)$ space) beyond the input array?

Stability: Does the algorithm preserve the relative order of elements with equal keys?

Adaptivity: Does the algorithm run faster if the input is already partially sorted?

Stability Example

Input: [(Alice, 10), (Bob, 5), (Charlie, 10)] sorted by score.

- Stable Sort: [(Bob, 5), (Alice, 10), (Charlie, 10)] (Alice remains before Charlie).
- Stability is crucial when sorting data with multiple attributes sequentially.

The Comparison Model and Lower Bounds

The Comparison Model

Algorithms that only use pairwise comparisons ($\leqslant, <, >, \geqslant$) to determine the order. This includes Merge Sort, Quick Sort, Heap Sort, etc.

The Theoretical Limit

Any comparison-based sorting algorithm must perform $\mathcal{O}(N \log N)$ comparisons in the worst case to sort N items.

Proof Sketch (Decision Tree Argument)

1. There are $N!$ possible permutations (orderings) of the input.
2. The algorithm must distinguish between these using binary comparisons (a decision tree). The tree must have at least $N!$ leaves.

The Comparison Model and Lower Bounds

The Comparison Model

Algorithms that only use pairwise comparisons ($\leqslant, <, >, \geqslant$) to determine the order. This includes Merge Sort, Quick Sort, Heap Sort, etc.

The Theoretical Limit

Any comparison-based sorting algorithm must perform $\mathcal{O}(\mathrm{N} \log \mathrm{N})$ comparisons in the worst case to sort N items.

Proof Sketch (Decision Tree Argument)

1. There are $\mathrm{N}!$ possible permutations (orderings) of the input.
2. The algorithm must distinguish between these using binary comparisons (a decision tree). The tree must have at least $\mathrm{N}!$ leaves.

The Comparison Model and Lower Bounds

Proof Sketch (Decision Tree Argument)

1. There are $N!$ possible permutations (orderings) of the input.
2. The algorithm must distinguish between these using binary comparisons (a decision tree). The tree must have at least $N!$ leaves.
3. The height of the tree H represents the worst-case number of comparisons.
4. A binary tree of height H has at most 2^H leaves.
5. $2^H \geqslant N! \implies H \geqslant \log_2(N!)$.
6. Using Stirling's approximation, $\log_2(N!) \approx N \log_2 N$.
7. Thus, $H = \mathcal{O}(N \log N)$.

The Comparison Model and Lower Bounds

Proof Sketch (Decision Tree Argument)

1. There are $N!$ possible permutations (orderings) of the input.
2. The algorithm must distinguish between these using binary comparisons (a decision tree). The tree must have at least $N!$ leaves.
3. The height of the tree H represents the worst-case number of comparisons.
4. A binary tree of height H has at most 2^H leaves.
5. $2^H \geqslant N! \implies H \geqslant \log_2(N!)$.
6. Using Stirling's approximation, $\log_2(N!) \approx N \log_2 N$.
7. Thus, $H = \mathcal{O}(N \log N)$.

The Comparison Model and Lower Bounds

Proof Sketch (Decision Tree Argument)

1. There are $N!$ possible permutations (orderings) of the input.
2. The algorithm must distinguish between these using binary comparisons (a decision tree). The tree must have at least $N!$ leaves.
3. The height of the tree H represents the worst-case number of comparisons.
4. A binary tree of height H has at most 2^H leaves.
5. $2^H \geqslant N! \implies H \geqslant \log_2(N!)$.
6. Using Stirling's approximation, $\log_2(N!) \approx N \log_2 N$.
7. Thus, $H = \mathcal{O}(N \log N)$.

The Comparison Model and Lower Bounds

Proof Sketch (Decision Tree Argument)

1. There are $N!$ possible permutations (orderings) of the input.
2. The algorithm must distinguish between these using binary comparisons (a decision tree). The tree must have at least $N!$ leaves.
3. The height of the tree H represents the worst-case number of comparisons.
4. A binary tree of height H has at most 2^H leaves.
5. $2^H \geqslant N! \implies H \geqslant \log_2(N!)$.
6. Using Stirling's approximation, $\log_2(N!) \approx N \log_2 N$.
7. Thus, $H = \mathcal{O}(N \log N)$.

The Comparison Model and Lower Bounds

Proof Sketch (Decision Tree Argument)

1. There are $N!$ possible permutations (orderings) of the input.
2. The algorithm must distinguish between these using binary comparisons (a decision tree). The tree must have at least $N!$ leaves.
3. The height of the tree H represents the worst-case number of comparisons.
4. A binary tree of height H has at most 2^H leaves.
5. $2^H \geqslant N! \implies H \geqslant \log_2(N!)$.
6. Using Stirling's approximation, $\log_2(N!) \approx N \log_2 N$.
7. Thus, $H = \mathcal{O}(N \log N)$.

The Comparison Model and Lower Bounds

Proof Sketch (Decision Tree Argument)

1. There are $N!$ possible permutations (orderings) of the input.
2. The algorithm must distinguish between these using binary comparisons (a decision tree). The tree must have at least $N!$ leaves.
3. The height of the tree H represents the worst-case number of comparisons.
4. A binary tree of height H has at most 2^H leaves.
5. $2^H \geqslant N! \implies H \geqslant \log_2(N!)$.
6. Using Stirling's approximation, $\log_2(N!) \approx N \log_2 N$.
7. Thus, $H = \mathcal{O}(N \log N)$.

The Comparison Model and Lower Bounds

Proof Sketch (Decision Tree Argument)

1. There are $N!$ possible permutations (orderings) of the input.
2. The algorithm must distinguish between these using binary comparisons (a decision tree). The tree must have at least $N!$ leaves.
3. The height of the tree H represents the worst-case number of comparisons.
4. A binary tree of height H has at most 2^H leaves.
5. $2^H \geqslant N! \implies H \geqslant \log_2(N!)$.
6. Using Stirling's approximation, $\log_2(N!) \approx N \log_2 N$.
7. Thus, $H = \mathcal{O}(N \log N)$.

Merge Sort: Divide and Conquer

The Divide and Conquer Paradigm

To achieve the optimal $\mathcal{O}(N \log N)$ complexity, we use Divide and Conquer.

The Strategy

1. **Divide:** Break the problem into smaller subproblems of the same type.
2. **Conquer:** Solve the subproblems recursively. Base case: solve directly if small enough.
3. **Combine:** Combine the solutions to the subproblems.

Merge Sort (John von Neumann, 1945)

1. **Divide:** Split the list A into two halves, L and R.
2. **Conquer:** Recursively sort L and R.
3. **Combine:** Merge the two sorted halves.

The Divide and Conquer Paradigm

To achieve the optimal $\mathcal{O}(\mathrm{N} \log \mathrm{N})$ complexity, we use Divide and Conquer.

The Strategy

1. **Divide:** Break the problem into smaller subproblems of the same type.
2. **Conquer:** Solve the subproblems recursively. Base case: solve directly if small enough.
3. **Combine:** Combine the solutions to the subproblems.

Merge Sort (John von Neumann, 1945)

1. **Divide:** Split the list A into two halves, L and R.
2. **Conquer:** Recursively sort L and R.
3. **Combine:** Merge the two sorted halves.

The Core Operation: Merging

The efficiency relies entirely on the ‘merge‘ operation.

The Merge Problem

Given two sorted lists, L1 and L2, combine them into a single sorted list M.

The Algorithm (Two-Pointer Technique)

1. Maintain pointers (indices) to the start of L1 and L2.
2. Compare the elements at the current pointers.
3. Append the smaller element to M.
4. Advance the pointer of the list from which the element was taken.
5. Repeat until one list is exhausted, then append the remainder of the other list.

Complexity of Merge

The Core Operation: Merging

The efficiency relies entirely on the ‘merge‘ operation.

The Merge Problem

Given two sorted lists, L1 and L2, combine them into a single sorted list M.

The Algorithm (Two-Pointer Technique)

1. Maintain pointers (indices) to the start of L1 and L2.
2. Compare the elements at the current pointers.
3. Append the smaller element to M.
4. Advance the pointer of the list from which the element was taken.
5. Repeat until one list is exhausted, then append the remainder of the other list.

Complexity of Merge

The Core Operation: Merging

The efficiency relies entirely on the ‘merge‘ operation.

The Algorithm (Two-Pointer Technique)

1. Maintain pointers (indices) to the start of L1 and L2.
2. Compare the elements at the current pointers.
3. Append the smaller element to M.
4. Advance the pointer of the list from which the element was taken.
5. Repeat until one list is exhausted, then append the remainder of the other list.

Complexity of Merge

The merge operation takes time proportional to the combined size of the lists.
$\mathcal{O}(|\mathrm{L1}| + |\mathrm{L2}|) = \mathcal{O}(\mathrm{N})$.

Merge Sort Implementation: The Recursive Structure (I)

We implement this using a generalised approach that accepts a comparison function.

```
def mergesort(lst, compare):
    # Base Case: Lists of size 0 or 1 are sorted
    if len(lst) <= 1:
        return lst
    # Divide: Find the midpoint
    pivot = len(lst) // 2
    # Conquer: Recursively sort the sublists
    # Note: Python slicing lst[:pivot] creates copies (O(N) time/
        space)
    lst1 = mergesort(lst[:pivot], compare)
    lst2 = mergesort(lst[pivot:], compare)
    # Combine: Merge the sorted sublists
    return merge(lst1, lst2, compare)
```

Merge Sort Implementation: The Recursive Structure (II)

We implement this using a generalised approach that accepts a comparison function.

Higher-Order Functions

‘mergesort‘ is a higher-order function because it accepts another function (‘compare‘) as an argument, making the sort generic.

Merge Implementation: The Linear Time Combine

```
1  def merge(lst1, lst2, compare):
2      merged_list = []
3      index1, index2 = 0, 0
4      len1, len2 = len(lst1), len(lst2)
5      while index1 < len1 or index2 < len2:
6          # Check if lst2 is exhausted OR
7          # (lst1 is not exhausted AND element in lst1 is "smaller")
8          if index2 == len2 or \
9             (index1 < len1 and compare(lst1[index1], lst2[index2])):
10             merged_list.append(lst1[index1])
11             index1 += 1
12         else:
13             merged_list.append(lst2[index2])
14             index2 += 1
15     return merged_list
```

Critical Analysis: Stability

Let's analyse the stability of the provided implementation.

Stability Requirement

If elements are equal, the element from the left list (`lst1`) must be chosen first to maintain the original relative order.

Analysing the Code Logic

```
15 if index2 == len2 or \
16    (index1 < len1 and compare(lst1[index1], lst2[index2])):
17     # Take from lst1
18 else:
19     # Take from lst2
```

Assume 'compare(a, b)' implements $a < b$. If $a = b$, 'compare' returns `False`. The 'if'

Critical Analysis: Stability

Let's analyse the stability of the provided implementation.

Analysing the Code Logic

```
15 if index2 == len2 or \
16    (index1 < len1 and compare(lst1[index1], lst2[index2])):
17     # Take from lst1
18 else:
19     # Take from lst2
```

Assume 'compare(a, b)' implements $a < b$. If $a = b$, 'compare' returns `False`. The 'if' condition (assuming neither list is exhausted) is `False`. The code executes the 'else' block, taking the element from `lst2`.

Conclusion

The provided implementation of `merge` is **NOT** stable if the `compare` function implements

Critical Analysis: Stability

Analysing the Code Logic

```
15 if index2 == len2 or \
16    (index1 < len1 and compare(lst1[index1], lst2[index2])):
17     # Take from lst1
18 else:
19     # Take from lst2
```

Assume ‘compare(a, b)‘ implements $a < b$. If $a = b$, ‘compare‘ returns `False`. The ‘if‘ condition (assuming neither list is exhausted) is `False`. The code executes the ‘else‘ block, taking the element from `lst2`.

Conclusion

The provided implementation of `merge` is **NOT** stable if the `compare` function implements strict inequality ($<$).

Merge Sort Analysis: The Recurrence Relation

We analyze the time complexity $T(N)$ using a recurrence relation.

The Relation

$T(N) =$ Time to divide $+$ Time to conquer $+$ Time to combine

- Divide: $\mathcal{O}(1)$ (calculating the pivot).
- Conquer: $T(N/2) + T(N/2) = 2T(N/2)$.
- Combine (Merge): $\mathcal{O}(N)$.

(Note: Python list slicing also takes $\mathcal{O}(N)$ time, but this doesn't change the overall recurrence structure).

The recurrence relation is:

$$T(N) = 2T(N/2) + \mathcal{O}(N)$$

$$T(1) = \mathcal{O}(1)$$

Merge Sort Analysis: The Recurrence Relation

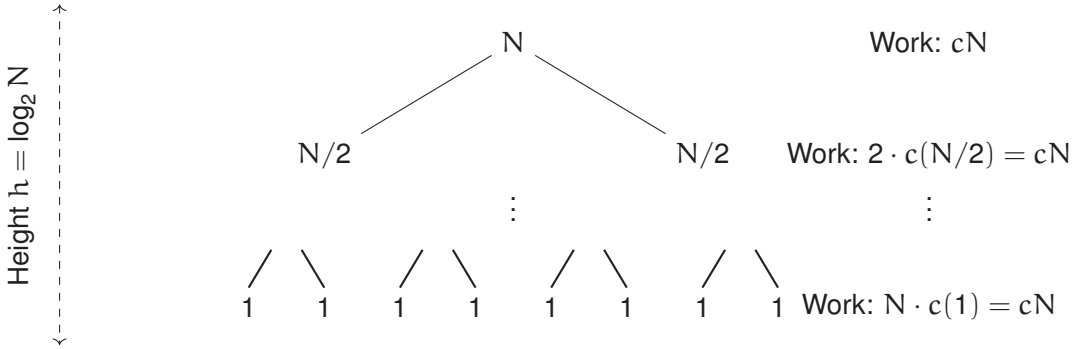
We analyze the time complexity $T(N)$ using a recurrence relation.

Solving the Recurrence

This standard recurrence can be solved using the Master Theorem (Case 2) or the Recursion Tree method.

Merge Sort Analysis: The Recursion Tree (I)

Visualising the work done at each level of recursion.



Merge Sort Analysis: The Recursion Tree (II)

Total Work

Total time = (Work per level) $\times$ (Number of levels)

$$T(N) = cN \times \log_2 N = \mathcal{O}(N \log N)$$

Conclusion

Merge Sort achieves the optimal lower bound for comparison-based sorting in the worst case.

Merge Sort Analysis: The Recursion Tree (II)

Total Work

Total time = (Work per level) $\times$ (Number of levels)

$$T(N) = cN \times \log_2 N = \mathcal{O}(N \log N)$$

Conclusion

Merge Sort achieves the optimal lower bound for comparison-based sorting in the worst case.

Properties of Merge Sort

Summary

Time Complexity: $\mathcal{O}(N \log N)$ (Best, Average, Worst cases). Performance very consistent.

Space Complexity: $\mathcal{O}(N)$ auxiliary space. Merge operation requires temporary storage.

In-Place: **No**.

Stability: **Yes** (if implemented correctly).

Comparison with Quick Sort

- Quick Sort is typically faster in practice (smaller constant factors) and is often in-place ($\mathcal{O}(\log N)$ space).
- However, Quick Sort has a worst-case time complexity of $\mathcal{O}(N^2)$ and is typically unstable.

Properties of Merge Sort

Summary

Time Complexity: $\mathcal{O}(N \log N)$ (Best, Average, Worst cases). Performance very consistent.

Space Complexity: $\mathcal{O}(N)$ auxiliary space. Merge operation requires temporary storage.

In-Place: **No**.

Stability: **Yes** (if implemented correctly).

Comparison with Quick Sort

- Quick Sort is typically faster in practice (smaller constant factors) and is often in-place ($\mathcal{O}(\log N)$ space).
- However, Quick Sort has a worst-case time complexity of $\mathcal{O}(N^2)$ and is typically unstable.

Generalisation and Higher-Order Functions

Abstraction and Higher-Order Functions

The implementation of ‘mergesort‘ highlights powerful concepts of abstraction and functional programming.

Higher-Order Functions (HOFs)

A function that takes one or more functions as arguments or returns a function as its result. Python treats functions as first-class citizens.

‘mergesort(lst, compare)‘

‘mergesort‘ is a HOF. It abstracts the comparison logic away from the sorting mechanism. This separates the *policy* (how to compare) from the *mechanism* (the sorting structure).

Abstraction and Higher-Order Functions

The implementation of ‘mergesort‘ highlights powerful concepts of abstraction and functional programming.

Higher-Order Functions (HOFs)

A function that takes one or more functions as arguments or returns a function as its result. Python treats functions as first-class citizens.

‘mergesort(lst, compare)‘

‘mergesort‘ is a HOF. It abstracts the comparison logic away from the sorting mechanism. This separates the *policy* (how to compare) from the *mechanism* (the sorting structure).

Using Custom Comparators

The generalised implementation allows for flexible sorting criteria.

Example 1: Case-Insensitive String Sorting

```
1 def cmp_str(s1, s2):
2     # Defines the ordering relation
3     return s1.lower() < s2.lower()
4
5 lst = ["BA", "aa", "bbbb", "a"]
6 sorted_lst = mergesort(lst, cmp_str)
7 # sorted_lst == ["a", "aa", "BA", "bbbb"]
```

Using Lambda Functions

Lambda functions provide a concise way to define anonymous functions inline, ideal for simple comparators.

Example 2: Sorting by Length

```
lst = ["BAAAAA", "aa", "bbbb", "a"]
# Define the comparator inline using a lambda
# Syntax: lambda arguments: expression
sorted_lst = mergesort(lst, lambda x, y: len(x) < len(y))
# sorted_lst == ["a", "aa", "bbbb", "BAAAAA"]
```

Python's Approach: Keys vs Comparators

Comparators vs Keys

- **Comparator (Our implementation):** Takes (A, B) and compares them. Called $\mathcal{O}(N \log N)$ times.
- **Key (Python's 'sorted()'):** Takes (A) and returns a value (the key) used for comparison. Called $\mathcal{O}(N)$ times.

Python's 'sorted()'

Python's Approach: Keys vs Comparators (cont.)

```
1 data = ["BAAAAA", "aa", "bbbb", "a"]
2 # Sort by length using the key parameter (more efficient)
3 result = sorted(data, key=len)
```

Efficiency

The 'key' approach is generally preferred in Python as it is more efficient due to fewer function calls.

Practical Considerations: Timsort

Python's Built-in Sort

Python's default sorting algorithm (`list.sort()` and `sorted()`) is **Timsort**.

Timsort: A Hybrid Approach

Timsort is a hybrid stable sorting algorithm, derived from Merge Sort and Insertion Sort, designed to perform well on real-world data.

- It is adaptive: exploits existing order (runs) in the data.
- Best Case: $\mathcal{O}(N)$ (if data is mostly sorted).
- Worst Case: $\mathcal{O}(N \log N)$.

Summary

Lecture Summary

Data Structures and ADTs

- **ADTs vs. Data Structures:** Separation of interface (what) from implementation (how).
- **Queues (FIFO):** Essential ADT.
- **Implementation:** Linked lists (with head and tail pointers) provide optimal $\mathcal{O}(1)$ Enqueue and Dequeue operations. Careful handling of edge cases (empty queue invariants) is critical.

Algorithms

Lecture Summary (cont.)

- **Complexity:** The lower bound for comparison sorts is $\mathcal{O}(\mathrm{N} \log \mathrm{N})$.
- **Divide and Conquer:** Powerful algorithmic paradigm.
- **Merge Sort:** An asymptotically optimal $\mathcal{O}(\mathrm{N} \log \mathrm{N})$ sorting algorithm. Requires $\mathcal{O}(\mathrm{N})$ space. Stability depends on the implementation details of the merge step.
- **Generalisation:** Higher-order functions (comparators) allow for flexible, generic algorithms.